

Doc. No.:	P0061R1
Date:	2015-10-23
Reply to:	Clark Nelson
	Richard Smith

__has_include for C++17

Replace paragraph 1 of 16.1 with several paragraphs, as follows:

defined-macro-expression:

defined *identifier*

defined (*identifier*)

h-preprocessing-token:

any *preprocessing-token* other than >

h-pp-tokens:

h-preprocessing-token

h-pp-tokens h-preprocessing-token

has-include-expression:

has_include (<*h-char-sequence*>)

has_include ("*q-char-sequence*")

has_include (*string-literal*)

has_include (< *h-pp-tokens* >)

The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below¹⁴⁶ and it may contain zero or more *defined-macro-expressions* and/or *has-include-expressions* as unary operator expressions, of the form

~~defined~~ *identifier*

~~or~~

~~defined (~~ *identifier* ~~)~~

which evaluate A *defined-macro-expression* evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has been the subject of a #define preprocessing directive without an intervening #undef directive with the same subject identifier), 0 if it is not.

The third and fourth forms of *has-include-expression* are considered only if neither of the first or second forms matches, in which case the preprocessing tokens are processed just as in normal text.

The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a #include directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a #include directive, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

The #ifdef and #ifndef directives, and the defined conditional inclusion operator, shall treat has_include as if it were the name of a defined macro. The identifier has_include shall not appear in any context not mentioned in this section.

Change 16.1p4:

... After all replacements due to macro expansion and the defined unary operator evaluations of *defined-macro-expressions* and *has-include-expressions* have been performed, all remaining identifiers and keywords¹⁴⁷, except for true and false, are replaced with the pp-number 0, and then each preprocessing token is converted into a token. ...

Add the following example after 16.1p6:

[Example: This demonstrates a way to include a library optional facility only if it is available.

```
#if __has_include(<optional>)
# include <optional>
```

```
# define have_optional 1
#elif __has_include(<experimental/optional>)
# include <experimental/optional>
# define have_optional 1
# define experimental_optional 1
#else
# define have_optional 0
#endif
```

-- end example 1